Pascal — Reference

A **Turbo-Pascal-flavoured Pascal** subset compiled with `lex` + `yacc` + `cc` (`pascal.l` / `pascal.y` lower Pascal to C; `cc` lowers the C to .NET IL). The program body becomes `main`; every top-level `procedure` / `function` becomes a `public static` method `p_<name>` on `CProgram`. Translation is syntax-directed: the parser carries a type table and emits C as it reduces, so records become C `struct`s, `string` becomes a 256-byte buffer, and `object` types become structs with a vtable pointer at offset 0.

Pascal is **strongly, statically typed** and **case-insensitive** (identifiers are folded to lower case, so `MyVar` and `myvar` are the same variable). It was designed to be LALR(1)-parseable, which is why it maps so directly onto this toolchain.

```
pascal prog.pas                # compile to prog.exe (native .NET executable)
pascal prog.pas -o app.exe     # choose the output name
```

Quick Reference

```
program Demo;                          { comments: { } or (* *) or // to end of line }
uses MathUtil;                         { inlines MathUtil.pas from the same directory }
const
  Max = 10;                            { integer / real / string / char constants }
  Pi  = 3.14159;
type
  Color  = (Red, Green, Blue);         { enumeration: ordered named constants }
  Grade  = 1..5;                       { subrange }
  TPoint = record x, y: integer end; { record }
  TVec   = array[1..5] of integer;     { array, arbitrary lower bound }
  PNode  = ^TPoint;                    { pointer }
var
  n: integer;  x: real;  c: char;  ok: boolean;  s: string;
  f: text;                             { text file }
  letters: set of char;                { set (0..255 bitset) }

function Square(k: integer): integer;         { result assigned to the function name }
begin Square := k * k end;
procedure Swap(var a: integer; var b: integer);   { var = by reference }
var t: integer;
begin t := a; a := b; b := t end;

begin
  n := 42;  s := 'text';  s := s + ' more';   { + concatenates strings }
  writeln('n = ', n, ' x = ', x:8:3);         { :width:decimals }
  if n > 0 then ... else ...;
  case n of 1: ...; 2, 3: ...; 4..6: ...; else ... end;
  for n := 1 to 10 do ...;   for n := 10 downto 1 do ...;
  while n > 0 do ...;        repeat ... until n = 0;
  with p do writeln(x, y);   { unqualified field names }
  if c in letters then ...
end.
```

Data types

`integer` (32-bit), `real` (64-bit double), `char`, `boolean` (`true`/`false`), `string` (a fixed 256-byte buffer), and `text` / `file` (a text file handle). **Enumerations** (`type Color = (Red, Green, Blue)`) are ordered constants starting at 0. **Subranges** (`1..5`) behave as integers. **Arrays** take arbitrary bounds and any number of dimensions (`array[1..3, 1..3] of integer`), indexed `a[i]` or `a[i, j]`; the element type may be `string` (`array[1..4] of string`). **Records** (`record … end`) nest, assign by value, and pass by value. **Pointers** are `^T`, allocated with `new` and released with `dispose`; `p^` dereferences, `@x`

takes an address. **Sets** (`set of char`) are 0..255 bitsets referenced by a runtime handle; a `[…]` literal is an ordinary expression, so `['a'..'z'] - vowels` and `[1, 2] + s` work as well as `c in ['a'..'z']`.

Literals: `123`, `1.5`, `1.5e3`, `'a string'` (double `''` for an embedded quote), and `'x'`. A one-character literal is a `char`, but it is also accepted wherever a `string` is expected — `s + '!'`, `s < 'Q'`, `s := 'A'`, and `concat(s, '!')` all do the string thing.

Statements / Commands

Assignment (`:=`), `if/then/else`, `case … of … else … end` (labels may be single values, comma lists, or `lo..hi` ranges), `while … do`, `for i := a to b do`, `for i := a downto b do`, `repeat … until`, `with rec do`, `goto` with numeric `label` declarations, `begin … end` compounds, and procedure calls.

Functions

Top-level `procedure` (no result) and `function` (result assigned to the function's own name, as in `Square := k * k`). Parameters are by value by default — including arrays and strings, which are copied on entry, so writing to them does not touch the caller's variable. A `var` parameter is by reference, so `Swap(x, y)` exchanges the caller's variables; `var` works for arrays and strings too (`procedure Fill(var a: IntArr)`). Recursion works.

`procedure P( … ); forward;` (or `function F( … ): T; forward;`) declares the signature only and emits a C prototype; the body is written later in the usual form. That is how mutual recursion between two subprograms is expressed.

Objects. `type T = object … end` declares an object type with fields and `procedure` / `function` / `constructor` / `destructor` members; `object(Parent)` gives single inheritance. A member marked `virtual` gets a vtable slot; a `constructor` installs the vtable pointer, after which calls dispatch dynamically — including through a `^TParent` pointer. Method bodies are written `procedure T.M( … )`, and inside them unqualified names resolve to fields and methods of the receiver.

Units. `unit Name; interface … implementation … end.` in a file next to the program; `uses Name;` inlines its declarations before the program body (whole-program inlining, not separate compilation).

Built-in functions. `ord`, `chr`, `abs`, `sqr`, `sqrt`, `sin`, `cos`, `exp`, `ln`, `trunc`, `odd`, `succ`, `pred`, `length`, `upcase`, `pos`, `copy`, `concat`, `eof`. **Built-in procedures.** `write`, `writeln`, `read`, `readln`, `inc`, `dec`, `new`, `dispose`, `halt`, and the file set `assign`, `reset`, `rewrite`, `append`, `close`.

Input / Output

`writeln(a, b, …)` writes its arguments and a newline; `write` omits the newline; bare `writeln` just ends the line. Each argument may carry Pascal's field specifiers: `x:width` and, for reals, `x:width:decimals`. Booleans print as `TRUE` / `FALSE`; reals without `:w:d` use `%g` formatting.

`readln(v, …)` reads whitespace-separated values from standard input into its arguments.

Text files: `var f: text;` then `assign(f, 'name.txt')`, `rewrite` (create) / `reset` (read) / `append`, `writeln(f, …)`, `readln(f, line)`, `eof(f)`, `close(f)`.

Graphics

None. Pascal here is a general-purpose teaching/systems language; Activity 8 draws with text.

Notes

- **Case-insensitive** throughout: `WriteLn`, `writeln`, and `WRITELN` are one identifier.
- `=` is equality, `<>` inequality, `:=` assignment; `div` is integer division, `mod` the remainder (truncated, as in Turbo Pascal: `-7 mod 3` is `-1`), and `/` always yields a real.
- `and`, `or`, `not` are boolean on booleans and bitwise on integers.
- Comments come in three forms: `{ … }`, `(* … *)`, and `//` to end of line.
- String comparison uses `strcmp`, so `<` and `>` order strings lexicographically.
- A qualified method call used as a *statement* needs parentheses (`obj.Show()`); used as an *expression* a paramless method may be written bare (`obj.Area`).

Subset boundaries

A broad Turbo-Pascal core, with these honest gaps and defects:

- **No nested procedures or functions.** A subprogram's local declarations are limited to `const`, `var`, `type`, and `label`.
- **Sets support `+`, `-`, `*`, and `in` only** — no `=`, `≤`, or `≥` between sets (those compare runtime handles, not membership). Sets are handles into a runtime table, so `b := a` aliases rather than copies.
- **A `string` is a fixed 256-byte buffer with no bounds check**, so a `var string` parameter or a long concatenation can overrun it.
- Field widths are honoured for integers and reals but **ignored for strings and chars**.
- Text files only — no typed or binary files, no `get` / `put`, no `seek`.
- Not implemented: variant records, `packed`, procedural parameters, `absolute`, sized strings (`string[20]`), open array parameters, `exit` / `break` / `continue`, exceptions, `try`, run-time range or overflow checking, separate compilation (units are inlined), and the wider Turbo Pascal unit library (`Crt`, `Dos`, `Graph`).
- The driver emits an executable only — there is no `--dll` flag, so there is no C#/VB.NET interop activity here (see `ada/ada.md` or `lua/lua.md` for languages that have one).

Tutorial

Every example was compiled and run with `pascal`; the output shown is real.

1. Your first program

```
program Hello;
begin
  writeln('Hello from Pascal')
end.
```

```
Hello from Pascal
```

`program Name;` is the header, the `begin … end.` block is the entry point (it becomes `main`), and the final `.` ends the program. Statements are separated by `;` — the one before `end` is optional.

2. Variables and data types

```
program Vars;
const
  Max = 10;
  Pi  = 3.14159;
type
  Color = (Red, Green, Blue);
var
  n: integer;
  x: real;
  c: char;
  ok: boolean;
  s: string;
  col: Color;
begin
  n := 42;  x := 2.5;  c := 'A';  ok := true;
  s := 'Pascal';  col := Green;
  writeln('n  = ', n);
  writeln('x  = ', x, '   x:8:3 = ', x:8:3);
  writeln('c  = ', c, '   ok = ', ok);
  writeln('s  = ', s, ' (length ', length(s), ')');
  writeln('col = ', ord(col));
  writeln('Max * 2 = ', Max * 2, '   Pi = ', Pi:0:2);
  writeln('7 div 2 = ', 7 div 2, '   7 / 2 = ', 7 / 2:0:2);
  writeln('sqrt(2) = ', sqrt(2):0:4, '   sqr(5) = ', sqr(5), '   abs(-3) = ', abs(-3))
end.
```

```
n  = 42
x  = 2.5   x:8:3 =    2.500
c  = A   ok = TRUE
s  = Pascal (length 6)
col = 1
Max * 2 = 20   Pi = 3.14
7 div 2 = 3   7 / 2 = 3.50
sqrt(2) = 1.4142   sqr(5) = 25   abs(-3) = 3
```

`const` values are fixed, `type` introduces names, `var` declares storage. `ord(Green)` is `1` — enumeration literals are ordered from 0. Note the two divisions: `div` truncates to an integer, `/` always produces a real. Without `:w:d` a real prints in `%g` style (`2.5`), which is why the formatted forms are used above.

3. Flow control

```
program Flow;
var i, sum: integer;
begin
  sum := 0;
  for i := 1 to 10 do sum := sum + i;
  writeln('for 1..10 sum = ', sum);
  for i := 3 downto 1 do write(i, ' ');
  writeln;
  i := 1;
  while i < 100 do i := i * 2;
  writeln('while doubling = ', i);
  i := 0;
  repeat i := i + 1 until i ≥ 3;
  writeln('repeat ended at ', i);
  for i := 1 to 4 do
    case i of
      1:    writeln(i, ': one');
      2, 3: writeln(i, ': two or three');
    else
      writeln(i, ': other');
    end
end.
```

```
for 1..10 sum = 55
3 2 1
while doubling = 128
repeat ended at 3
1: one
2: two or three
3: two or three
4: other
```

`for … to` counts inclusively, `downto` counts back. `while` tests before the body, `repeat … until` after it. `case` labels may be single values, comma-separated lists, or `lo..hi` ranges, with `else` as the catch-all.

4. Structured types — arrays, records and sets

Pascal's contribution to programming languages was the *structured type*:

```
program Structured;
type
  TPoint = record x, y: integer end;
  TGrid  = array[1..3, 1..3] of integer;
var
  a: array[1..5] of integer;
  names: array[1..2] of string;
  m: TGrid;
  p, q: TPoint;
  vowels, cons: set of char;
  i, j: integer;
begin
  for i := 1 to 5 do a[i] := i * i;
  write('squares:');
  for i := 1 to 5 do write(' ', a[i]);
  writeln;

  for i := 1 to 3 do
    for j := 1 to 3 do m[i, j] := i * j;
  for i := 1 to 3 do
  begin
    for j := 1 to 3 do write(m[i, j]:4);
    writeln
  end;

  p.x := 3; p.y := 4;
  q := p;                       { records assign by value }
  q.x := 9;
  writeln('p = (', p.x, ',', p.y, ')   q = (', q.x, ',', q.y, ')');
  with p do writeln('with p: x=', x, ' y=', y);

  names[1] := 'ada';
  names[2] := 'niklaus';
  writeln('names: ', names[1], ' & ', names[2]);

  vowels := ['a', 'e', 'i', 'o', 'u'];
  cons   := ['a'..'z'] - vowels;      { a [...] literal is an ordinary operand }
  if 'b' in cons   then writeln('b is a consonant');
  if 3 in [1, 2, 4..6] then writeln('3 in set') else writeln('3 not in set')
end.
```

```
squares: 1 4 9 16 25
   1   2   3
   2   4   6
   3   6   9
p = (3,4)   q = (9,4)
with p: x=3 y=4
names: ada & niklaus
b is a consonant
3 not in set
```

Array bounds are whatever you declare — `array[1..5]` really starts at 1, and the element type may be `string`. `q := p` copies the whole record, which is why changing `q.x` leaves `p` alone. `with p do` brings `p`'s field names into scope. A `[…]` set literal is an ordinary expression, so `['a'..'z'] - vowels` needs no intermediate variable; `x in […]` still compiles to an inline comparison chain rather than building a set.

5. Subroutines and functions

```
program Subs;
var a, b: integer;

function Square(k: integer): integer;
begin
  Square := k * k
end;

function Fact(k: integer): integer;
begin
  if k ≤ 1 then Fact := 1
  else Fact := k * Fact(k - 1)
end;

procedure Swap(var x: integer; var y: integer);
var t: integer;
begin
  t := x;  x := y;  y := t
end;

procedure Rule(w: integer);
var i: integer;
begin
  for i := 1 to w do write('-');
  writeln
end;

begin
  writeln('Square(7) = ', Square(7));
  writeln('Fact(6)   = ', Fact(6));
  a := 1;  b := 2;
  Swap(a, b);
  writeln('after Swap: a=', a, ' b=', b);
  Rule(12)
end.
```

```
Square(7) = 49
Fact(6)   = 720
after Swap: a=2 b=1
------------
```

A `function` returns a value by assigning to its own name (`Square := k * k`); a `procedure` returns nothing. `var` parameters are by reference, so `Swap` really exchanges `a` and `b`; without `var`, even an

array or `string` argument is copied on entry. Recursion (`Fact`) works because each top-level subprogram becomes an ordinary static method, and `forward` lets two subprograms call each other:

```
program Parity;
var n: integer;
procedure IsOdd(n: integer); forward;
procedure IsEven(n: integer);
begin
  if n = 0 then writeln('even') else IsOdd(n - 1)
end;
procedure IsOdd(n: integer);
begin
  if n = 0 then writeln('odd') else IsEven(n - 1)
end;
begin
  for n := 0 to 4 do begin write(n, ': '); IsEven(n) end
end.
```

```
0: even
1: odd
2: even
3: odd
4: even
```

6. Memory management

Variables are static or stack storage; the heap is explicit, via `new` and `dispose`:

```
program Memory;
type
  TNode = record
    value: integer;
    next:  ^TNode;
  end;
  PNode = ^TNode;
var
  head, n, t: PNode;
  i: integer;
begin
  head := nil;
  for i := 3 downto 1 do
  begin
    new(n);                  { heap-allocate one TNode }
    n^.value := i * 10;
    n^.next  := head;
    head := n
  end;
  n := head;
  while n <> nil do
  begin
    writeln('node ', n^.value);
    n := n^.next
  end;
  n := head;
  while n <> nil do
  begin
    t := n^.next;
    dispose(n);              { and give it back }
    n := t
  end;
  writeln('list freed')
end.
```

```
node 10
node 20
node 30
list freed
```

`new(p)` allocates one instance of `p`'s pointed-to type, `p^` dereferences it, and `dispose(p)` frees it. `nil` is the null pointer. Underneath, the allocation is `malloc` into `cc`'s flat byte arena, so a Pascal pointer is an integer offset — which is what makes it expressible in verifiable IL.

7. Strings

`string` is a 256-byte buffer, `+` concatenates, and characters are 1-based:

```
program Strings;
var s, t, u: string;
    i: integer;
begin
  s := 'Pascal';
  t := ' rules';
  u := s + t;
  writeln(u);
  writeln('length      = ', length(u));
  writeln('copy(u,1,6) = ', copy(u, 1, 6));
  writeln('pos(''cal'', s) = ', pos('cal', s));
  writeln('s[1] = ', s[1], '  s[6] = ', s[6]);
  for i := length(s) downto 1 do write(s[i]);
  writeln;
  if s = 'Pascal' then writeln('compares equal');
  writeln('upcase(s[2]) = ', upcase(s[2]));
  writeln('s + ''!''      = ', s + '!');
  if s < 'Q' then writeln('s sorts before ''Q''')
end.
```

```
Pascal rules
length      = 12
copy(u,1,6) = Pascal
pos('cal', s) = 4
s[1] = P  s[6] = l
lacsaP
compares equal
upcase(s[2]) = A
s + '!'      = Pascal!
s sorts before 'Q'
```

`copy(s, start, count)` extracts a substring (1-based), `pos(sub, s)` finds one (0 if absent), `s[i]` yields a `char`. Note `''` inside a literal is one apostrophe. A one-character literal such as `'!'` is a `char`, but in a string context it is widened to a one-character string, so `s + '!'` and `s < 'Q'` behave as Turbo Pascal does.

8. Drawing a picture — a text diamond

```
program Diamond;
const N = 4;
var row: integer;

procedure Line(pad: integer; stars: integer);
var k: integer;
begin
  for k := 1 to pad do write(' ');
  for k := 1 to stars do write('*');
  writeln
end;

begin
  for row := 1 to N do Line(N - row, 2 * row - 1);
  for row := N - 1 downto 1 do Line(N - row, 2 * row - 1)
end.
```

```
   *
  ***
 *****
*******
 *****
  ***
   *
```

One helper procedure emits `pad` spaces then `stars` asterisks; the top half counts up and the bottom half counts `downto`.

9. Objects and virtual methods

Turbo Pascal's `object` type is a record with methods; `virtual` adds dynamic dispatch through a vtable pointer stored at offset 0 (which is why a derived object is layout- compatible with its base):

```
program Virt;
type
  TShape = object
    name: string;
    constructor Init(n: string);
    function Area: integer; virtual;
    procedure Show;
  end;
  TSquare = object(TShape)
    side: integer;
    constructor Init(s: integer);
    function Area: integer; virtual;
  end;
  TCircle = object(TShape)
    r: integer;
    constructor Init(rad: integer);
    function Area: integer; virtual;
  end;

constructor TShape.Init(n: string);
begin name := n end;
function TShape.Area: integer;
begin Area := 0 end;
procedure TShape.Show;
begin writeln(name, ' area = ', Area) end;

constructor TSquare.Init(s: integer);
begin name := 'square'; side := s end;
function TSquare.Area: integer;
begin Area := side * side end;

constructor TCircle.Init(rad: integer);
begin name := 'circle'; r := rad end;
function TCircle.Area: integer;
begin Area := 3 * r * r end;

var sq: TSquare; ci: TCircle; p: ^TShape;
begin
  sq.Init(5);
  ci.Init(10);
  sq.Show();
  ci.Show();
  p := @sq;  writeln('via base pointer: ', p^.Area);
  p := @ci;  writeln('via base pointer: ', p^.Area)
end.
```

```
square area = 25
circle area = 300
via base pointer: 25
via base pointer: 300
```

`TShape.Show` is *not* virtual and is never overridden, yet it prints 25 then 300: the `Area` it calls is dispatched through the object's vtable, installed by whichever `constructor` ran. Assigning `@sq` to a `^TShape` and calling `p^.Area` shows the same dispatch through a base-typed pointer. Note that a method call used as a statement needs parentheses (`sq.Show()`), while a paramless method in an expression does not (`p^.Area`).

10. Units and text files

A `unit` groups declarations; `uses` inlines it. Combined here with the text-file procedures:

```
{ MathUtil.pas }
unit MathUtil;
interface
function Square(x: integer): integer;
function Cube(x: integer): integer;
implementation
function Square(x: integer): integer;
begin Square := x * x end;
function Cube(x: integer): integer;
begin Cube := x * x * x end;
end.
```

```
program Report;
uses MathUtil;
var f: text;
    line: string;
    i: integer;
begin
  assign(f, 'report.txt');
  rewrite(f);
  for i := 1 to 4 do
    writeln(f, i, ' ', Square(i), ' ', Cube(i));
  close(f);

  assign(f, 'report.txt');
  reset(f);
  while not eof(f) do
  begin
    readln(f, line);
    writeln('read: ', line)
  end;
  close(f)
end.
```

```
read: 1 1 1
read: 2 4 8
read: 3 9 27
read: 4 16 64
```

`MathUtil.pas` sits next to `Report.pas`; the compiler reads it, drops the interface signatures, and splices the implementation in ahead of the program body — whole-program inlining rather than separate compilation. `assign` / `rewrite` / `close` write the file, `assign` / `reset` / `readln` / `eof` / `close` read it back a line at a time.

From here the natural extensions are nested procedures and real separate compilation of units (see *Subset boundaries*).